

Architecture Decision Record: Adopting ASP.NET Core Identity

Status

@ramy jamea

Context

The Restaurant Management System requires a secure and scalable authentication and authorization solution to manage multiple user types such as Administrators, Branch Managers, Cashiers, and Employees.

The original implementation used a custom `User` model with:

- a manually managed password hash,
- a `RoLe` enum,
- and custom authentication logic.

As the system grows, this approach becomes difficult to maintain, less secure, and harder to scale.

To address these limitations, the system adopts ASP.NET Core Identity as the standard membership and authentication framework.

Decision

The system will use ASP.NET Core Identity for:

- user authentication,
- password management,
- role management,
- claims-based authorization,

- and integration with JWT authentication.

Identity entities such as:

- `IdentityUser`,
- `IdentityRole`,
- and related Identity tables

will replace the previous custom authentication structure.

Reasons for the Decision

1. Improved Security

ASP.NET Core Identity provides industry-standard security features out-of-the-box, including:

- secure password hashing and salting,
- protection against brute-force attacks,
- account lockout mechanisms,
- and secure token generation.

This removes the need to implement sensitive security logic manually.

2. Scalable Role Management

The previous `Role` enum limited users to a fixed set of roles and required code changes whenever new roles were introduced.

Identity stores roles in database tables such as:

- `AspNetRoles`
- `AspNetUserRoles`

This allows roles to be added or modified dynamically without changing application code.

3. Support for Multiple Roles

The custom implementation allowed only one role per user.

Identity supports many-to-many relationships between users and roles, enabling a user to hold multiple permissions simultaneously when required by business needs.

4. Claims-Based Authorization

Identity supports claims-based authentication.

Claims such as:

- `BranchId` ,
- `Role` ,
- or `UserId`

can be embedded inside JWT tokens during login.

This reduces repeated database queries and improves API performance.

5. Reduced Development Complexity

Building a complete authentication system manually requires:

- password security implementation,
- token management,
- role validation,
- failed login tracking,
- and authorization logic.

Identity provides these features through a well-tested framework, reducing development time and maintenance effort.

Consequences

Positive Consequences

- Stronger application security
 - Faster authentication development
 - Easier role and permission management
 - Better scalability for future business requirements
 - Standardized authentication architecture familiar to .NET developers
 - Simplified JWT integration
-

Negative Consequences

- Additional Identity tables increase database complexity
 - Developers must understand the Identity framework structure
 - Customization may require learning Identity internals
-

Conclusion

Adopting ASP.NET Core Identity provides a secure, scalable, and maintainable authentication solution for the Restaurant Management System.

The decision reduces security risks, simplifies future development, and aligns the project with modern .NET development standards.